Chapter 9. Commands

This chapter covers

- Understanding the role of commands in the unified log
- Modeling commands
- Using Apache Avro to define schemas for commands
- Processing commands in our unified log

So far, we have concerned ourselves almost exclusively with events. Events, as we explained in chapter 1, are discrete occurrences that take place at a specific point in time. Throughout this book, we have created events, written them to our unified log, read them from our unified log, validated them, enriched them, and more. There is little that we have not done with events!

But we can represent another unit of work in our unified log: the *command*. A command is an order or instruction for a specific action to be performed in the future—each command, when executed, produces an event. In this chapter, we will show how representing commands explicitly as one or more streams within our unified log is a powerful processing pattern.

We will start with a simple definition for commands, before moving on to show how decision-making apps can operate on events in our unified log and express decisions in the form of commands. Modeling commands for maximum flexibility and utility is something of an art; we will cover this next before launching into the chapter's applied example.

Working for Plum, our fictitious global consumer-electronics manufacturer, we will define a command intended to alert Plum maintenance engineers of overheating machines on the factory floor. We will represent this command in the Apache Avro schema language and use the Kafka producer script from chapter 2 to publish those alert commands to our unified log.

We will then write a simple command-executor app by using the Kafka Java client's producer and consumer capabilities. Our command executor will read the Kafka topic containing our alert commands and send an email to the appropriate support engineer for each alert; we will use Rackspace's Mailgun transactional email service to send the emails.

Finally, we will wrap up the chapter with some design questions to consider before adding commands into your company's own unified log. We will consider the pros and cons of two designs for your command topic(s) in Kafka or Kinesis and will also introduce the idea of a hierarchy of commands.

Let's get started!

9.1. Commands and the unified log

What is a command exactly, and what does it mean in the context of the unified log? In this section, we'll introduce commands as complementary to events, and argue for making decision-making apps, and the commands they generate, a central part of your unified log.

9.1.1. Events and commands

A *command* is an order or instruction for a specific action to be performed in the future. Here are some example commands:

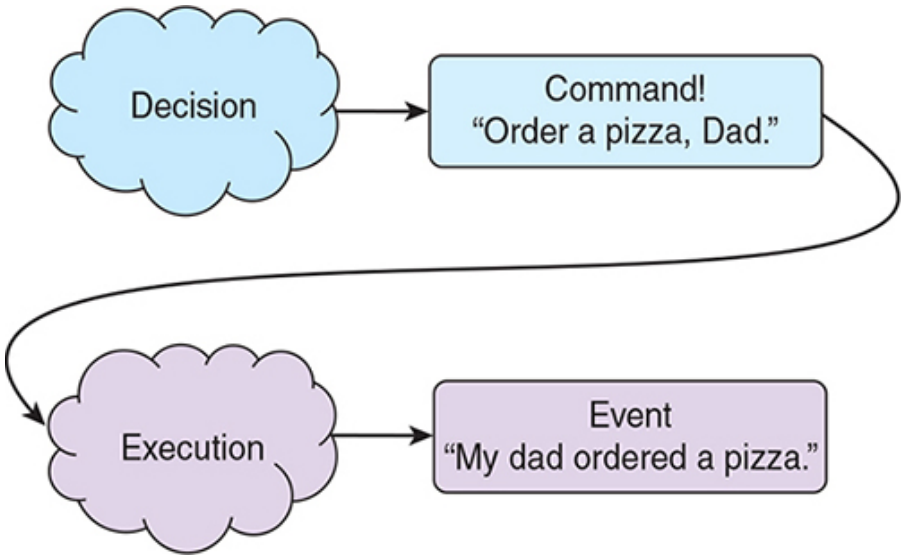
- "Order a pizza, Dad."
- "Tell my boss that I quit."
- "Renew our household insurance, Jackie."

If an event is a record of an occurrence in *the past*, then a command is the expression of intent that a new event will occur *in the future*:

- My dad ordered a pizza.
- I quit my job.
- Jackie renewed our household insurance.

Figure 9.1 illustrates the behavioral flow underpinning the first example: a *decision* (I want pizza) produces a *command* ("Order a pizza, Dad"), and if or when that command is *executed*, then we can record an event as having taken place (my dad ordered a pizza). In grammatical terms: an event is a *past-tense* verb in the *indicative mood*, whereas a command is a verb in the *imperative mood*.

Figure 9.1. A decision produces a command—an order or instruction to do something specific. If that command is then executed, we can record an event as having occurred.



You can see that there is a symbiotic relationship between commands and events: strictly speaking, without commands, we would have no events.

9.1.2. Implicit vs. explicit commands

If behind every event is a command, how could we have reached chapter 9 in a book all about events without encountering a single command? The answer is that almost all software relies on *implicit commands*—the decision to do something is immediately followed by the execution of that decision, all in the same block of code. Here is a simple example of this in pseudocode:

```
function decide_and_act(obj):
    if obj is an apple:
        eat the apple
        emit_event(we ate the apple)
    else if obj is a song:
        sing the song
        emit_event(we sang the song)
    else if obj is a beer:
        drink the beer
        emit_event(we drank the beer)
    else:
        emit_event(we don't recognize the obj)
```

This code includes no explicit commands: instead, we have a block of code that mixes decision-making in with the execution of those decisions. How would this code look if we had explicit commands? Something like this:

```
function make_decision(obj):
    if obj is an apple:
        return command(eat the apple)
    else if obj is a song:
```

```
        return command(sing the song)
    else if obj is a beer:
        return command(drink the beer)
    else:
        return command(complain we don't recognize the obj)
```

Of course, this version is not functionally identical to the previous code block. In this version, we are only *returning commands* as the result of our decisions; we are not actually executing those commands. There will have to be another piece of code downstream of `make_decision(obj)`, looking something like this:

```
function execute_command(cmd):
    if cmd is eat the apple:
        eat the apple
        emit_event(we ate the apple)
    else if cmd is sing the song:
        sing the song
        emit_event(we sang the song)
    else if cmd is drink the beer:
        drink the beer
        emit_event(we drank the beer)
    else if cmd is complain we don't recognize the obj
        emit_event(we don't recognize the obj)
```

It looks like we have turned something simple—making a decision and acting on it—into something more complicated: making a decision, emitting a command, and then executing that command. The reason we do this is in support of *separation of concerns*.[1] Turning our commands into explicit, first-class entities brings major advantages:

1

You can learn more about separation of concerns at Wikipedia: https://en.wikipedia.org/wiki/Separation_of_concerns.

- ***It makes our decision-making code simpler.*** All `make_decision(obj)` needs to be able to do is emit a command. It doesn't need to understand the mechanics of eating apples or singing songs; nor does it need to know how to track events.
- ***It makes our decision-making code easier to test.*** We have to test only that our code makes the right decision, not that it then executes on that decision correctly.
- ***It makes our decision-making process more auditable.*** All decisions lead to concrete commands that can be reviewed. By contrast,

with `decide_and_act(obj)`, we have to explore the *outcomes* of the actions to understand what decisions were made.
- ***It makes our execution code more DRY ("don't repeat yourself").*** We can implement eating an apple in a single code location, and any code that decides to eat an apple emits a command instructing the eating.
- ***It makes our execution code more flexible.*** If our command is "send Jenny an email," we can swap out one transactional email service provider (for example, Mandrill) for another (for example, Mailgun or SendGrid).
- ***It makes our execution code repeatable.*** We can replay a sequence of commands in the same order as they were issued at any time if the context requires it.

This decoupling of decision-making and command execution clearly has benefits, but if we involve our unified log, things get even more powerful. Let's explore this next.

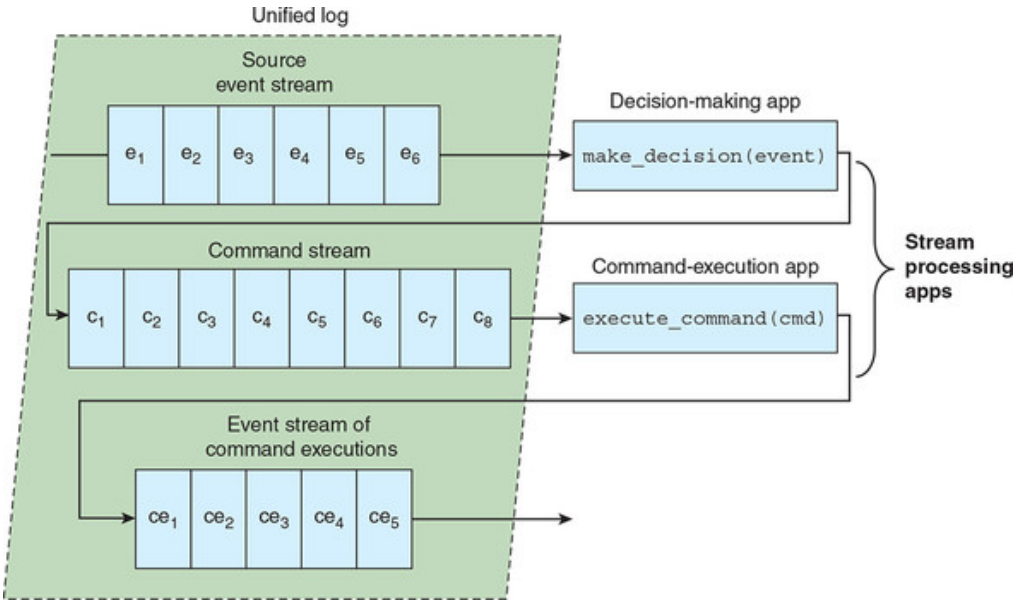
9.1.3. Working with commands in a unified log

In the previous section, we split a single function, `decide_and_act(obj)`, into two separate functions:

- `make_decision(obj)`—Made a decision based on the supplied `obj` and returned a command to execute
- `execute_command(cmd)`—Executed the supplied command `cmd` and emitted an event recording the execution

With a unified log, there is no reason that these two functions need to live in the same application: we can create a new stream called *commands* and write the output of `make_decision(obj)` into that stream. We can then write a separate stream-processing job that consumes the *commands* stream and executes the commands found within by using `execute_command(cmd)`. This is visualized in figure 9.2.

Figure 9.2. Within our unified log, a source event stream is used to drive a decision-making app, which emits commands to a second stream. A command-execution app reads that stream of commands, executes the commands, and then emits a stream of events recording the command executions.



With this architecture, we have completely decoupled the decision-making process from the execution of the commands. Our stream of commands is now a *first-class entity*: we can attach whatever applications we want to execute (or even just observe) those commands.

In the next section, we will add commands to our unified log.

9.2. Making decisions

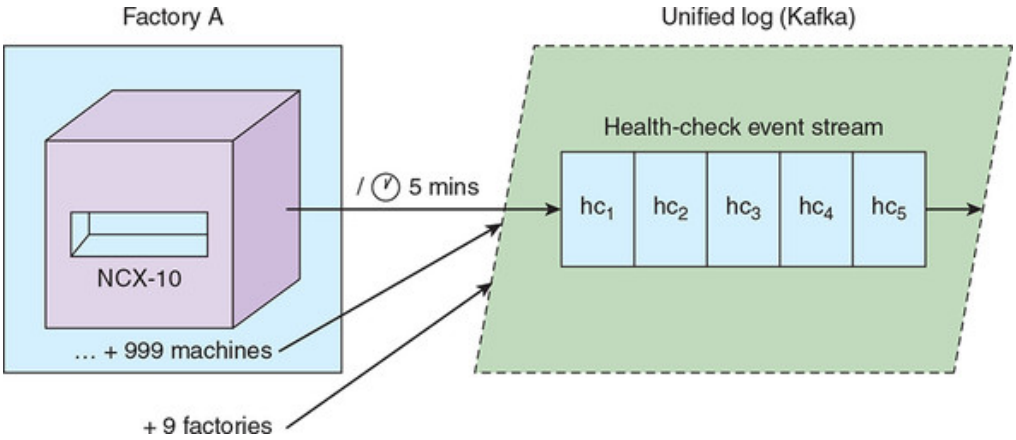
To generate commands, we'll first have to make some decisions. In this section, we'll continue working with Plum, our fictional company with a unified log introduced in chapter 7, and wire some decision-making into that unified log. Let's get started.

9.2.1. Introducing commands at Plum

Let's imagine again that we are in the BI team at Plum, a global consumer-electronics manufacturer. Plum is in the process of implementing a unified log; for unimportant reasons, the unified log is a hybrid, using Amazon Kinesis for certain streams and Apache Kafka elsewhere. In reality, this is a fairly unusual setup, but it enables us to work with both Kinesis and Kafka in this section of the book.

At the heart of the Plum production line is the NCX-10 machine, which stamps new laptops out of a single block of steel. Plum has 1,000 of these machines in each of its 10 factories. Each machine emits key metrics as a form of health check to a Kafka topic every 5 minutes, as shown in figure 9.3.

Figure 9.3. All of the NCX-10 machines in Plum's factories emit a standard health-check event to a Kafka topic every 5 minutes.



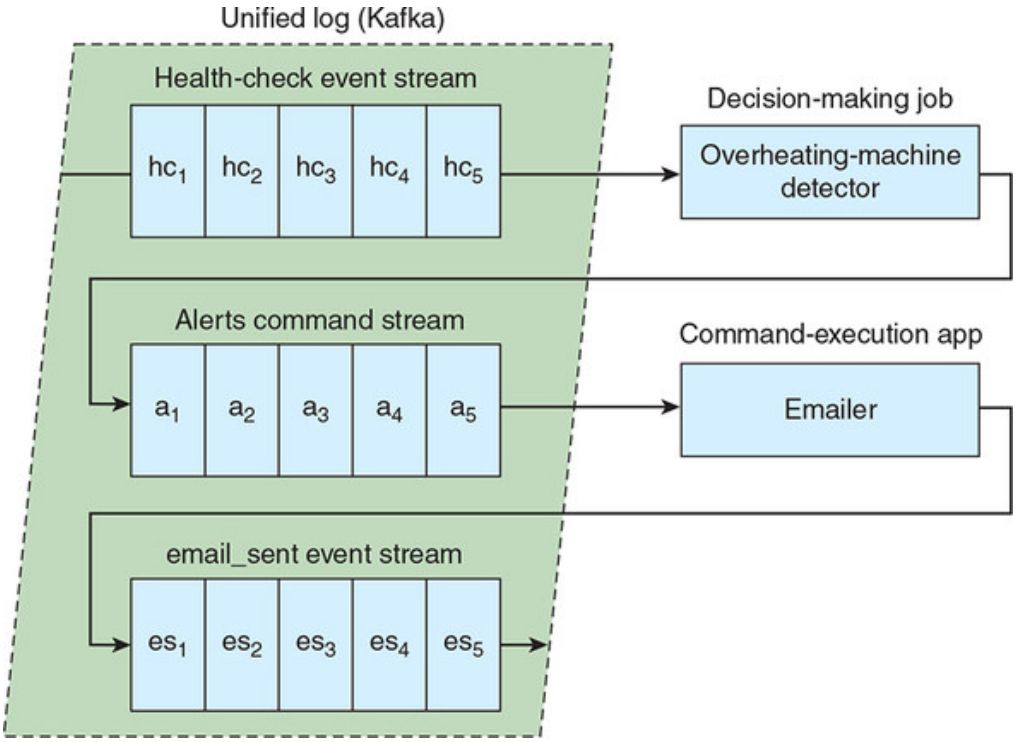
We have been asked by the plant maintenance team at Plum to use the health-check data to help keep the NCX-10 machines humming. The team members are particularly concerned about one scenario that they want to tackle first: if a machine shows signs of overheating, a maintenance engineer needs to be alerted immediately in order to be able to inspect the machine and fix any issues with the cooling system.

We can deliver what plant maintenance wants by writing two stream-processing jobs:

- ***The decision-making job***— This will parse the event stream of NCX-10 health checks to detect signs of machines overheating. If these are detected, this job will emit a command to alert a maintenance engineer.
- ***The command-execution job***— This will read the event stream containing commands to alert a maintenance engineer, and will send each of these alerts to the engineer.

Figure 9.4 depicts these two jobs and the Kafka streams between them.

Figure 9.4. Within Plum's unified log, a stream of health-check events is read by a decision-making app to detect overheating machines and emit alert commands to a second stream. A command-execution app reads the stream of commands and sends emails to alert the maintenance engineers.



Does the decision-making job sound familiar? It's likely a piece of either single or multiple (stateful) event processing, as we explored in part 1. Because this chapter is about commands, we'll skip over the implementation of the decision-making job and head straight to defining the command itself.

9.2.2. Modeling commands

Our decision-making job needs to emit a command to alert a maintenance engineer that a specific NCX-10 machine shows signs of overheating. It's important that we model this command correctly: it will represent the *contract* between the decision-making job, which will emit the command, and the command-executing job, which will read the command.

We know what this command needs to do—alert a maintenance engineer—but how do we decide what fields to put in our command? Like a tasty muffin, a good command should have these characteristics:

- ***Shrink-wrapped***— The command must contain everything that could possibly be required to execute the command; the executor should not need to look up additional information to execute the command.
- ***Fully baked***— The command should define exactly the action for the executor to perform; we should not have to add business logic into the executor to turn each command into something actionable.

Here is an example of a badly designed self-describing alert command for Plum:

```
{ "command": "alert_overheating_machine",
  "recipient": "Suzie Smith",
  "machine": "cz1-123",
  "createdAt": 1539576669992
}
```

Let's consider how the executor would have to be written for this command, in pseudocode:

```
function execute_command(cmd):
    if cmd.type == "alert_overheating_machine":
        email_address = lookup_email_address(cmd.recipient)
        subject = "Overheating machine"
        message = "Machine {{cmd.machine}} may be overheating!"
        send_email(email_address, subject, message)
        emit_event(alerted_the_maintenance_engineer)
```

Unfortunately, this breaks both of the rules for a good command:

- Our command executor has to look up a crucial piece of information, the engineer's email address, in another system.
- Our command executor has to include business logic to translate the command type plus the machine ID into an actionable alert.

Here is a better version of an alert command for Plum:

```
{ "command" : "alert",
    "notification": {
    "summary": "Overheating machine",
    "detail": "Machine cz1-123 may be overheating!",
    "urgency": "MEDIUM"
  },
  "recipient": {
    "name": "Suzie Smith",
    "phone": "(541) 754-3010",
    "email": "s.smith@plum.com"
  },
  "createdAt": 1539576669992
}
```

The command may be a little more verbose, but the executor implementation for this command is much simpler:

```
function execute_command(cmd):
    if cmd.command == "alert":
        send_email(cmd.recipient.email, cmd.notification.summary,
```

```
            cmd.notification.detail)
        emit_event(alerted_the_maintenance_engineer)
```

This command requires the executor to do much less work: the executor now only has to send the email to the recipient and record an event to the same effect. The executor does not need to know that this specific alert relates to an overheating machine in a factory; the specifics of this alert were "fully baked" upstream, in the business logic of the decision-making job. This strong decoupling has the major advantage of making our executor *general-purpose*. Plum can write other decision-making jobs that can reuse this command-executing job without requiring any code changes in the executor.

The other thing to note is that we have made the alert command *definitive* but not overly *restrictive*. If we wanted to, we could update our executor to include a prioritization system for processing alerts, like so:

```
function execute_command(cmd):
    if cmd.command == "alert":
        if cmd.urgency == "HIGH":
            send_sms(cmd.recipient.phone, cmd.notification.detail)
        else:
            send_email(cmd.recipient.email, cmd.notification.summary,
                cmd.notification.detail)
           emit_event(alerted_the_maintenance_engineer)
```

Using our unified log for commands makes updates like this achievable:

- We only have to update our command-executor job to add the prioritization logic. There's no need to update the decision-making jobs in any way.
- We can test the new functionality by replaying old commands through the new executor.
- We can switch over to the new executor gradually, potentially running it in parallel to the old executor and allocating only a portion of commands to the new executor.

9.2.3. Writing our alert schema

We are now ready to define a *schema* for our alert command. We are going to use Apache Avro (https://avro.apache.org) to model this schema. As explained in chapter 6, Avro schemas can be defined in plain JSON files. The schema file needs to live somewhere, so we'll add it into our command-executor app. We'll need to create this app next.

We are going to write the command executor as a Java application, called `ExecutorApp`, using Gradle. Let's get started. First, create a directory called plum. Then switch to that directory and run this:

```
$ gradle init --type java-library
...
BUILD SUCCESSFUL
...
```

Gradle has created a skeleton project in that directory, containing a couple of Java source files for stub classes, called Library.java and LibraryTest.java. Delete these two files, as we will be writing our own code shortly.

Next let's prepare our Gradle project build file. Edit the file build.gradle and replace its current contents with the following listing.

Listing 9.1. build.gradle

```
plugins {                                                    1
    id "java"
    id "application"
    id "com.commercehub.gradle.plugin.avro" version "0.8.0"
}

sourceCompatibility = '1.8'

mainClassName = 'plum.ExecutorApp'

repositories {
  mavenCentral()
}

version = '0.1.0'

dependencies {                                               2
  compile 'org.apache.kafka:kafka-clients:2.0.0'
  compile 'org.apache.avro:avro:1.8.2'
  compile 'net.sargue:mailgun:1.9.0'
  compile 'org.slf4j:slf4j-api:1.7.25'
}


jar {
  manifest {
    attributes 'Main-Class': mainClassName
  }
```

```
  from {
    configurations.compile.collect {
      it.isDirectory() ? it : zipTree(it)
    }
  } {
    exclude "META-INF/*.SF"
    exclude "META-INF/*.DSA"
    exclude "META-INF/*.RSA"
  }
}
```

- ***1*** **A Gradle plugin to generate Java code from Apache Avro schemas**
- ***2*** **Dependencies include Kafka and Avro**

Let's check that we can build this:

```
$ gradle compileJava
...
BUILD SUCCESSFUL
...
```

Now we are ready to work on the schema for our new command.

9.2.4. Defining our alert schema

To add our new schema for alerts to the project, create a file at this path:

```
src/main/resources/avro/alert.avsc
```

Populate this file with the Avro JSON schema in the following listing.

Listing 9.2. alert.avsc

```
{ "name": "Alert",
  "namespace": "plum.avro",
  "type": "record",
  "fields": [
    { "name": "command", "type": "string" },
    { "name": "notification",
      "type": {
        "name": "Notification",
        "namespace": "plum.avro",
        "type": "record",
        "fields": [
          { "name": "summary", "type": "string" },
          { "name": "detail", "type": "string" },
```

```
            { "name": "urgency",
              "type": {
                "type": "enum",
                "name": "Urgency",
                "namespace": "plum.avro",
                "symbols": ["HIGH", "MEDIUM", "LOW"]
              }
            }
          ]
        }
      },
      { "name": "recipient",
        "type": {
          "type": "record",
          "name": "Recipient",
          "namespace": "plum.avro",
          "fields": [
            { "name": "name", "type": "string" },
            { "name": "phone", "type": "string" },
            { "name": "email", "type": "string" }
          ]
        }
      },
      { "name": "createdAt", "type": "long" }
    ]
  }
```

Quite a lot is going on in this Avro schema file. Let's break it down:

- Our top-level entity is a record called `Alert` that belongs in the `plum.avro` namespace (as do all of our entities).
- Our `Alert` consists of a `type` field, a `Notification` child record, a `Recipient` child record, and a `createdAt` timestamp for the alert.
- Our `Notification` record contains `summary`, `detail`, and `urgency` fields, where `urgency` is an enum with three possible values.
- Our `Recipient` record contains `name`, `phone`, and `email` fields.

A minor piece of housekeeping—we need to soft-link the resources/avro subfolder to another location so that the Avro plugin for Gradle can find it:

```
$ cd src/main && ln -s resources/avro .
```

With our Avro schema defined, let's use the Avro plugin in our build.gradle file to automatically generate Java bindings for our schema:

```
$ gradle generateAvroJava
:generateAvroProtocol UP-TO-DATE
:generateAvroJava

BUILD SUCCESSFUL

Total time: 8.234 secs
...
```

You will find the generated files inside your Gradle build folder:

```
$ ls build/generated-main-avro-java/plum/avro/
Alert.java  Notification.java  Recipient.java  Urgency.java
```

These files are too lengthy to reproduce here, but if you open them in your text editor, you will see that the files contain POJOs to represent the three records and one enumeration that make up our alert.

This is a wrap for designing our alert command for Plum and modeling that command in Apache Avro. Next, we can proceed to building our command executor.

9.3. Consuming our commands

Imagine that Plum now has a Kafka topic containing a constant stream of our alert commands, all stored in Avro format. With this stream in place, Plum now needs to implement a command executor, which will consume those alerts and execute them. First, we will get all the plumbing in place and check that we can successfully deserialize the incoming Avro event.

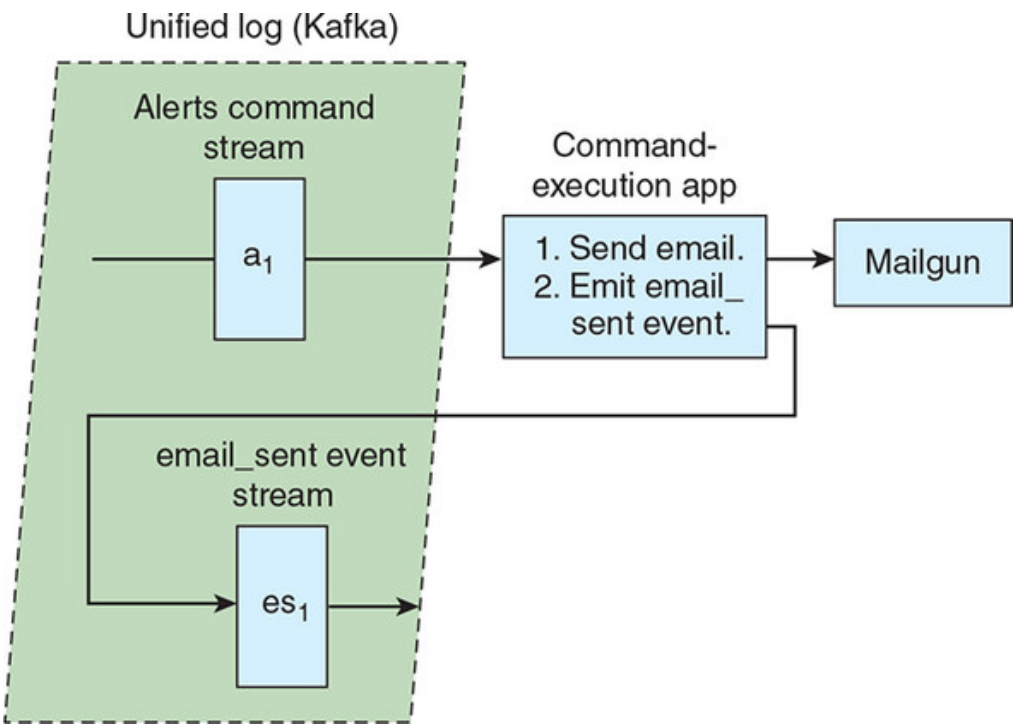
9.3.1. The right tool for the job

We could use lots of different stream-processing frameworks to execute our commands, but remember that command execution involves only two tasks:

1. Reading each command from the stream and executing it
2. Emitting an event to record that the command has been executed

Figure 9.5 shows the specifics of these two tasks for Plum.

Figure 9.5. The command-execution app for Plum will send an email to the support engineer via Mailgun and then emit an `email_sent` event to record that the alert has been executed.



Both tasks are performed on one command at a time; there's no need to consider multiple commands at once. Our command execution is analogous to the single-event processing we explored in chapter 2. As with single-event processing, we will need only a simple framework to execute our commands, so the simple consumer and producer capabilities of the Kafka Java client library from chapter 2 will fit the bill nicely.

9.3.2. Reading our commands

As a first step, we need to read in our individual commands as records from our Kafka topic `commands`. Remember from chapter 2 that this is called a *consumer* in Kafka parlance. As in that chapter, we will write our own consumer in Java, using the Kafka Java client library.

Let's create a file for our consumer, called src/main/java/plum/Consumer.-java, and add in the code in listing 9.3. This is a direct copy of the consumer code in chapter 2, except for the following changes:

- The new package name is `plum`.
- This code passes each consumed record to an `executor` rather than a `producer`.

Listing 9.3. Consumer.java

```
package plum;

import java.util.*;

import org.apache.kafka.clients.consumer.*;
```

```
public class Consumer {

  private final KafkaConsumer<String, String> consumer;
  private final String topic;

  public Consumer(String servers, String groupId, String topic) {
    this.consumer = new KafkaConsumer<String, String>(
      createConfig(servers, groupId));
    this.topic = topic;
  }

  public void run(IExecutor executor) {
    this.consumer.subscribe(Arrays.asList(this.topic));
    while (true) {
      ConsumerRecords<String, String> records = consumer.poll(100);
      for (ConsumerRecord<String, String> record : records) {
        executor.execute(record.value());

      }
    }
  }

  private static Properties createConfig(
    String servers, String groupId) {

    Properties props = new Properties();
    props.put("bootstrap.servers", servers);
    props.put("group.id", groupId);
    props.put("enable.auto.commit", "true");
    props.put("auto.commit.interval.ms", "1000");
    props.put("auto.offset.reset", "earliest");
    props.put("session.timeout.ms", "30000");
    props.put("key.deserializer",
      "org.apache.kafka.common.serialization.StringDeserializer");
    props.put("value.deserializer",
      "org.apache.kafka.common.serialization.StringDeserializer");
    return props;
  }
}
```

- ***1*** **The package is now plum.**
- ***2*** **In place of the producer, we now have an executor.**

So far, so good—we have defined a consumer that will read all the records from a given Kafka topic and hand them over to the `execute` method of the supplied executor. Next, we will implement an initial version of the executor. This won't yet carry out the command, but it will show that we can successfully parse our Avro-structured events into Java objects.

9.3.3. Parsing our commands

See how our consumer is going to run the `IExecutor.execute()` method for each incoming command? To keep things flexible, the two executors we write in this chapter will both conform to the `IExecutor` interface, letting us easily swap out one for the other. Let's now define this interface in another file, called src/main/java/plum/IExecutor.java. Add in the code in the following listing.

Listing 9.4. IExecutor.java

```
package plum;

import java.util.Properties;

import org.apache.kafka.clients.producer.*;

public interface IExecutor {

  public void execute(String message);          1

  public static void write(KafkaProducer<String, String> producer,
    String topic, String message) {
    ProducerRecord<String, String> pr = new ProducerRecord(
      topic, message);
    producer.send(pr);
  }

  public static Properties createConfig(String servers) {
    Properties props = new Properties();
    props.put("bootstrap.servers", servers);
    props.put("acks", "all");
    props.put("retries", 0);
    props.put("batch.size", 1000);
    props.put("linger.ms", 1);
    props.put("key.serializer",
      "org.apache.kafka.common.serialization.StringSerializer");
    props.put("value.serializer",
      "org.apache.kafka.common.serialization.StringSerializer");
    return props;
  }
}
```

- ***1* Our abstract execute method, for concrete implementations of IExecutor to instantiate**

Again, this code is extremely similar to the `IProducer` we wrote in chapter 2. As with that `IProducer`, this interface contains static helper meth-

ods to configure a Kafka record producer and write events to Kafka. We need these capabilities in our executor so that we can fulfill the second part of executing any given command: emitting an event back into Kafka and recording this command as having been successfully executed.

With the interface in place, let's write our first concrete implementation of `IExecutor`. At this stage, we won't execute the command, but we want to check that we can successfully parse the incoming command. Add the code in the following listing into a new file called src/main/java/plum/EchoExecutor.java.

Listing 9.5. EchoExecutor.java

```
package plum;

import java.io.*;

import org.apache.kafka.clients.producer.*;

import org.apache.avro.*;
import org.apache.avro.io.*;
import org.apache.avro.generic.GenericData;
import org.apache.avro.specific.SpecificDatumReader;

import plum.avro.Alert;

public class EchoExecutor implements IExecutor {

  private final KafkaProducer<String, String> producer;
  private final String eventsTopic;

  private static Schema schema;
  static {
    try {
      schema = new Schema.Parser()
        .parse(EchoExecutor.class.getResourceAsStream("/avro/alert.avsc
    } catch (IOException ioe) {
      throw new ExceptionInInitializerError(ioe);
    }
  }

  public EchoExecutor(String servers, String eventsTopic) {

    this.producer = new KafkaProducer(IExecutor.createConfig(servers));
    this.eventsTopic = eventsTopic;
  }

  public void execute(String command) {
```

```
      InputStream is = new ByteArrayInputStream(command.getBytes());
      DataInputStream din = new DataInputStream(is);

      try {
        Decoder decoder = DecoderFactory.get().jsonDecoder(schema, din);
        DatumReader<Alert> reader = new SpecificDatumReader<Alert>(schema
        Alert alert = reader.read(null, decoder);
        System.out.println("Alert " + alert.recipient.name + " about " +
          alert.notification.summary);
      } catch (IOException | AvroTypeException e) {
        System.out.println("Error executing command:" + e.getMessage());
      }
    }
  }
```

- ***1*** **The Gradle Avro plugin automatically generates this Alert POJO from the bundled schema.**
- ***2*** **Statically initialize this Schema object from the bundled schema.**
- ***3*** **Deserialize our alert command into an Alert POJO.**
- ***4*** **Print out salient information about the alert.**

The `EchoExecutor` is simple. Every time the `execute` method is invoked with a serialized command, it does the following:

1. Attempts to deserialize the incoming command into an `Alert` POJO, where that `Alert` POJO has been statically generated (by the Gradle Avro plugin) from the `alert.avsc` schema
2. If successful, prints the salient information about the alert out to `stdout`

9.3.4. Stitching it all together

We can now stitch these three files together via a new `ExecutorApp` class containing our `main` method. Create a new file called src/main/java/plum/ExecutorApp.java and populate it with the contents of the following listing.

Listing 9.6. ExecutorApp.java

```
package plum;

public class ExecutorApp {

  public static void main(String[] args){
    String servers       = args[0];
    String groupId       = args[1];
    String commandsTopic = args[2];
```

```
        String eventsTopic   = args[3];

        Consumer consumer = new Consumer(servers, groupId, commandsTopic);
        EchoExecutor executor = new EchoExecutor(servers, eventsTopic);
        consumer.run(executor);
      }
    }
```

We will pass four arguments into our `StreamApp` on the command line:

- `servers` specifies the host and port for talking to Kafka.
- `groupId` identifies our code as belonging to a specific Kafka consumer group.
- `commandsTopic` is the Kafka topic of commands to read from.
- `eventsTopic` is the Kafka topic we will write events to.

Let's build our stream processing app now. From the project root, the plum folder, run this:

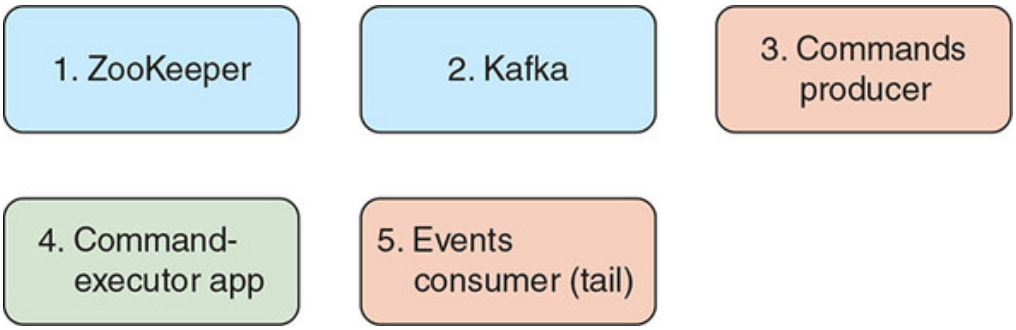
```
$ gradle jar
...
BUILD SUCCESSFUL

Total time: 25.532 secs
```

We are now ready to test our stream processing app.

9.3.5. Testing

To test our new application, we are going to need five terminal windows. Figure 9.6 sets out what we'll be running in each of these terminals.

Figure 9.6. The five terminals we need to run in order to test our command-executor app include ZooKeeper, Kafka, one command producer, the command executor app, and a consumer for the events emitted by the command executor.



Each of our first three terminal windows will run a shell script from inside our Kafka installation directory:

```
$ cd ~/kafka_2.12-2.0.0
```

In our first terminal, we start up ZooKeeper:

```
$ bin/zookeeper-server-start.sh config/zookeeper.properties
```

In our second terminal, we start up Kafka:

```
$ bin/kafka-server-start.sh config/server.properties
```

In our third terminal, let's start a script that lets us send commands into our `alerts` Kafka topic:

```
$ bin/kafka-console-producer.sh --topic alerts \
  --broker-list localhost:9092
```

Let's now give this producer an alert command, by pasting this into the same terminal, making sure to add a newline after the command to send it into the Kafka topic:

```
{ "command" : "alert", "notification": { "summary": "Overheating machin
 "detail": "Machine cz1-123 may be overheating!", "urgency": "MEDIUM" }
 "recipient": { "name": "Suzie Smith", "phone": "(541) 754-3010", "emai
 "s.smith@plum.com" }, "createdAt": 1539576669992 }
```

Phew! We are finally ready to start up our new command-executing application. In a fourth terminal, head back to your project root, the plum folder, and run this:

```
$ cd ~/plum
$ java -jar ./build/libs/plum-0.1.0.jar localhost:9092 ulp-ch09 \
  alerts events
```

This has kicked off our app, which will now read all commands from `alerts` and execute them. Wait a second and you should see the following output in the same terminal:

```
Alert Suzie Smith about Overheating machine
```

Good news: our simple `EchoExecutor` is working a treat. Now we can move onto the more complex version to execute the alert and log the sent

email. Shut down the stream processing app with Ctrl-Z and then type `kill %%`, but make sure to leave that terminal and the other terminal windows open for the next section.

9.4. Executing our commands

Now that we can successfully parse our alert commands from Avro-based JSONs into POJOs, we can now move on to *executing* those commands. In this section, we will wire in a third-party email service to send our alerts, and then finish up by emitting an event to record that the email has been sent. Let's get started.

9.4.1. Signing up for Mailgun

Our overlords at Plum want the monitoring alerts about the NCX-10 machines to be sent to the maintenance engineers via email. So, we need to integrate some kind of email-sending mechanism into our executor. We have many to choose from out there, but we will go with a hosted transactional email service called Mailgun (www.mailgun.com).

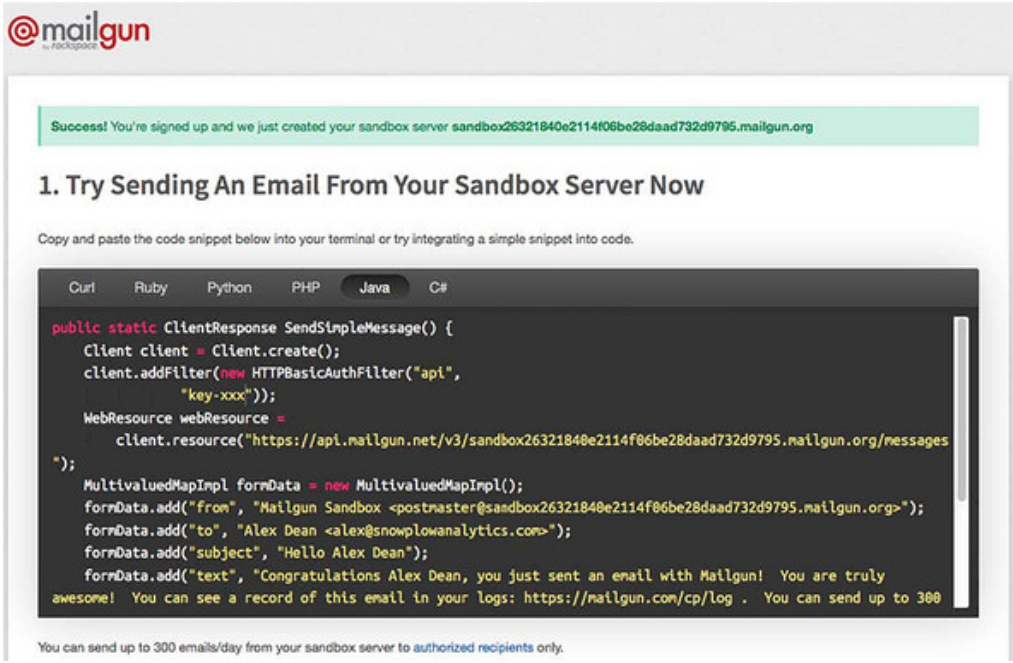
Mailgun lets us sign up for a new account and start sending emails without providing any billing information, which is perfect for this kind of experimentation. If you prefer, you could equally use an alternative provider like Amazon Simple Email Service (SES), SendGrid, or even a local mail server option such as Postfix. If you go with an alternative email service provider, you will need to adjust the following instructions accordingly.

Head to the signup page for Mailgun:
https://signup.mailgun.com/new/signup.

Fill in your details; you don't need to provide any billing information. Click the Create Account button, and on the next screen you will see some Java code for sending an email, as shown in figure 9.7. Before you can send an email, you have to do two more things:

1. Activate your Mailgun account. Mailgun will have sent a confirmation email to your signup email; click on this and follow the instructions to authenticate your account.
2. Add an authorized recipient. You can follow the link at the bottom of figure 9.7 to add an email recipient for testing. If you don't have a second email address, you can add *+ulp* or something similar to your existing address (for example, alex+ulp@foo.com). Again, you need to click on the confirmation email to authorize this.

Figure 9.7. Click the Java tab on this get-started screen from Mailgun and you'll see some basic code for sending an email via Mailgun from Java.



9.4.2. Completing our executor

With these steps completed, we are now ready to update our executor to send emails. First, we need a thin wrapper around the Mailgun email-sending functionality: create a file called src/main/java/plum/Emailer.java and add in the contents of the following listing.

Listing 9.7. Emailer.java

```
package plum;

import net.sargue.mailgun.*;

import plum.avro.Alert;

public final class Emailer {

  static final String MAILGUN_KEY =
    "XXX";                                          1
  static final String MAILGUN_SANDBOX =
    "sandboxYYY.mailgun.org";                       2

  private static final Configuration configuration = new Configuration(
    .domain(MAILGUN_SANDBOX)
    .apiKey(MAILGUN_KEY)
    .from("Test account", "postmaster@" + MAILGUN_SANDBOX);

  public static void send(Alert alert) {
    Mail.using(configuration)
```

```
        .to(alert.recipient.email.toString())
        .subject(alert.notification.summary.toString())
        .text(alert.notification.detail.toString())
        .build()
        .send();
    }
}
```

- ***1*** **Replace the XXX with your Mailgun API key.**
- ***2*** **Replace the YYY with your Mailgun sandbox server.**

Emailer.java contains a simple wrapper around the Mailgun emailing library that we are using. Make sure to update the constants with the API key and sandbox server details found in your Mailgun account.

With this in place, we can now write our full executor. Like the `EchoExecutor`, this will implement the `IExecutor` interface, but the `FullExecutor` will also send the email via Mailgun and log the event to our events topic in Kafka. Add the contents of the following listing into the file src/main/java/plum/FullExecutor.java.

Listing 9.8. FullExecutor.java

```
package plum;

import java.io.*;

import org.apache.kafka.clients.producer.*;

import org.apache.avro.*;
import org.apache.avro.io.*;
import org.apache.avro.generic.GenericData;
import org.apache.avro.specific.SpecificDatumReader;

import plum.avro.Alert;

public class FullExecutor implements IExecutor {

  private final KafkaProducer<String, String> producer;
  private final String eventsTopic;

  private static Schema schema;
  static {
    try {
      schema = new Schema.Parser()
        .parse(EchoExecutor.class.getResourceAsStream("/avro/alert.avsc
    } catch (IOException ioe) {
      throw new ExceptionInInitializerError(ioe);
```

```
    }
  }

  public FullExecutor(String servers, String eventsTopic) {

    this.producer = new KafkaProducer(IExecutor.createConfig(servers));
    this.eventsTopic = eventsTopic;
  }

  public void execute(String command) {

    InputStream is = new ByteArrayInputStream(command.getBytes());
    DataInputStream din = new DataInputStream(is);

    try {
      Decoder decoder = DecoderFactory.get().jsonDecoder(schema, din);
      DatumReader<Alert> reader = new SpecificDatumReader<Alert>(schema
      Alert alert = reader.read(null, decoder);
      Emailer.send(alert);                                    1
      IExecutor.write(this.producer, this.eventsTopic,
        "{ \"event\": \"email_sent\" }");                     2
    } catch (IOException | AvroTypeException e) {
      System.out.println("Error executing command:" + e.getMessage());
    }
  }
}
```

- *1* **Send our email via Mailgun.**
- *2* **Emit an event to record the event being sent.**

`FullExecutor` is similar to our earlier `EchoExecutor`, but with two additions:

- We are now sending the email to our support engineer via the new `Emailer`.
- Following the successful email send, we are logging a basic event to record that success to a Kafka topic containing events.

Now we need to rewrite the entry point to our app to use our new executor. Edit the file src/main/java/plum/ExecutorApp.java and repopulate it with the contents of the following listing.

Listing 9.9. ExecutorApp.java

```
package plum;

import java.util.Properties;
```

```
public class ExecutorApp {

  public static void main(String[] args){
    String servers       = args[0];
    String groupId       = args[1];
    String commandsTopic = args[2];
    String eventsTopic   = args[3];

    Consumer consumer = new Consumer(servers, groupId, commandsTopic);
    FullExecutor executor = new FullExecutor(servers, eventsTopic);
    consumer.run(executor);
  }
}
```

- ***1* Replaced the EchoExecutor with a FullExecutor**

Let's rebuild our app now. From the project root, the plum folder, run this:

```
$ gradle jar
...
BUILD SUCCESSFUL

Total time: 25.532 secs
```

We are now ready to rerun our command executor.

9.4.3. Final testing

Head back to the terminal running the previous build of the executor. If it is still running, kill it with Ctrl-C. Now restart it:

```
$ java -jar ./build/libs/plum-0.1.0.jar localhost:9092 ulp-ch09 \
  alerts events
```

Leave ZooKeeper and Kafka running in their respective terminals; we now need to start the fifth and final terminal, to tail our Kafka topic containing events:

```
$ bin/kafka-console-consumer.sh --topic events --from-beginning \
  --bootstrap-server localhost:9092
```

Now let's head back to the terminal that is running a producer connected to our `alerts` topic. We'll take the alert command we ran before, update

it so that the recipient's email is the same one that we authorized with Mailgun, and then paste it into the producer:

```
$ bin/kafka-console-producer.sh --topic alerts \
  --broker-list localhost:9092
{ "command" : "alert", "notification": { "summary": "Overheating machin
 "detail": "Machine cz1-123 may be overheating!", "urgency": "MEDIUM" }
 "recipient": { "name": "Suzie Smith", "phone": "(541) 754-3010", "emai
 "alex+test@snowplowanalytics.com" }, "createdAt": 1543392786232 }
```

For this to work, you *must* update the email in the alert to your own email, the one that you authorized with Mailgun. Check back in your new terminal that is tailing our Kafka events topic and you should see this:

```
$ bin/kafka-console-consumer.sh --topic events --from-beginning \
  --zookeeper localhost:2181
{ "event": "email_sent" }
```

That's encouraging; it suggests that our command executor has sent our email. Check in your email client and you should see an incoming email, something like figure 9.8.

Figure 9.8. Our command-executor app is now able to email alert commands to us via Mailgun.

Great! We have now implemented a command executor that can take incoming commands—in our case, alerts for a long-suffering Plum maintenance engineer—and convert them into emails to that engineer. Our command executor even emits an `email_sent` event to track that the action has been performed.

9.5. Scaling up commands

This chapter has walked you through a simple example of executing an alert command as it passes through Plum's unified log. So far, so good—but how do we scale this up to a real company with hundreds or thousands of possible commands? This section has some ideas.

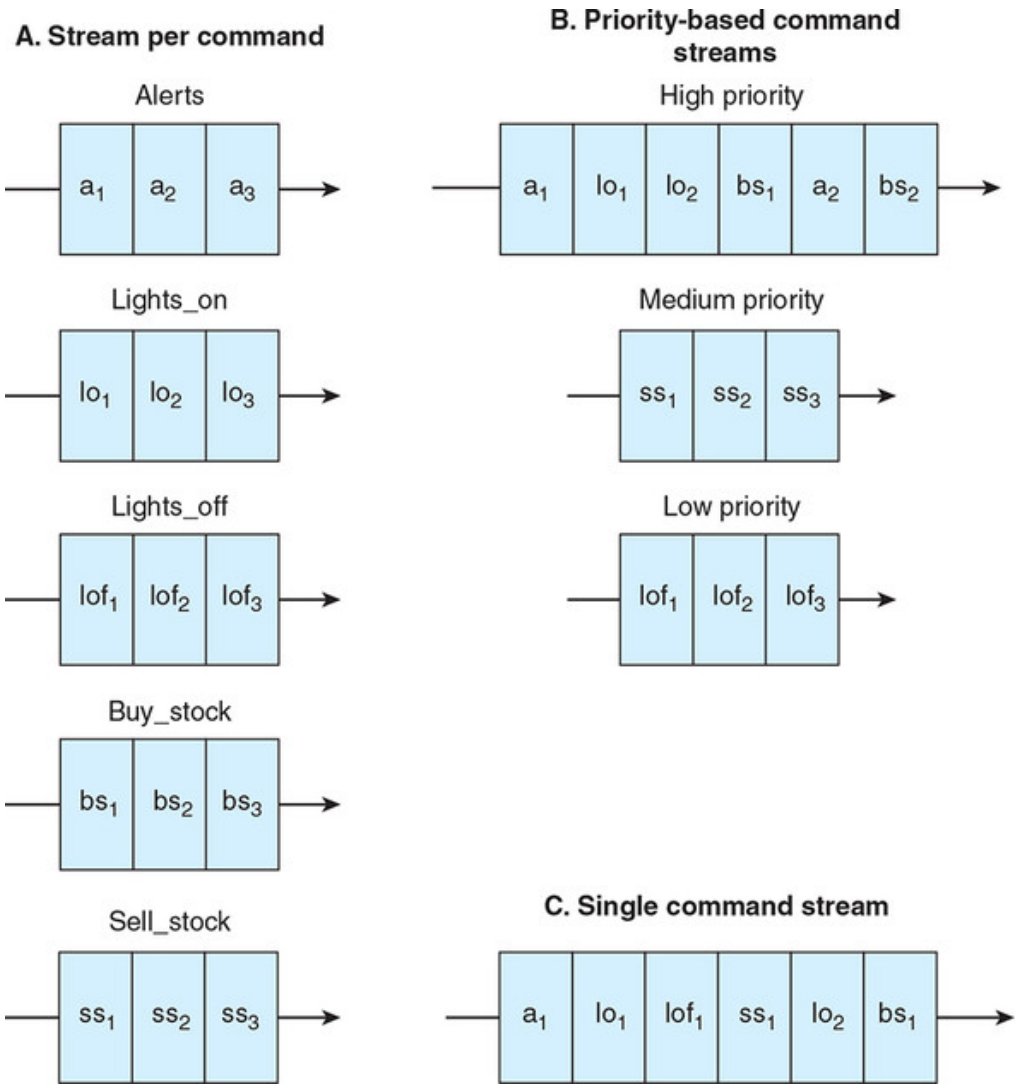
9.5.1. One stream of commands, or many?

In this chapter, we called our stream of commands `alerts` on the basis that it would contain only alert commands, and our command-executor app processed all incoming commands as alerts. If we want to extend our implementation to support hundreds or thousands of commands, we have several options:

- Have one stream (topic, in Kafka parlance) per command—in other words, hundreds or thousands of streams.
- Make the commands self-describing but write them to say, three or five streams; each stream represents a different command *priority*.
- Make the commands *self-describing* and have a single stream. Include a header in each record that tells the command executor what type of command this is.

Figure 9.9 illustrates these three options.

Figure 9.9. We could define one stream for each command type (option A), associate commands to priority-based streams (option B), or use one stream for all commands (option C).



When choosing one of these options, you will want to consider the operational overhead of having multiple streams versus the development over-

head of making your commands self-describing. It is also important to understand how these different setups fare in the face of *command-execution failures*.

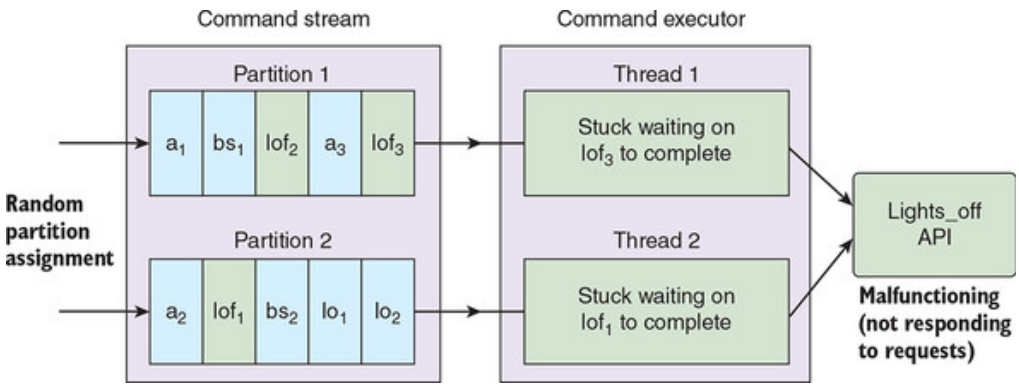
9.5.2. Handling command-execution failures

Imagine that Mailgun has an outage (whether planned or unplanned) for several hours. What will happen to our command executor? Remember that our command executor is performing single-event processing; it has no way of knowing that a systemic problem has arisen with Mailgun. If we are lucky, our Mailgun email-sending code will simply time out after perhaps 30 seconds, for each alert command that is processed. In this case, we would likely emit an Email Failed to Send event or similar back into Kafka.

Now imagine that our command executor is reading a stream containing many types of commands. The failure of each alert command costs us 30 seconds of processing time, so if our decision-making apps are generating new commands at a faster rate than this, our command executor will fall further and further behind. Other high-priority commands that Plum wants to execute will be extremely delayed.

Note that the sharded nature of a Kafka or Kinesis stream doesn't help you: assuming that the failing command type is distributed across all of the shards, the worker assigned to each shard is equally likely to suffer slowdowns as the other workers. This is illustrated in figure 9.10.

Figure 9.10. With a command stream containing two partitions, a single malfunctioning command can cause both threads within the command executor (one per partition) to get "stuck." Here, the `lights_off` command is failing to execute because of the unavailability of the Lights_off external API.



Separating the commands into streams with different priorities does not necessarily help either: it is still possible for a high-priority command type to slow other high-priority commands, as also seen in figure 9.10.

Two potential solutions to consider are as follows:

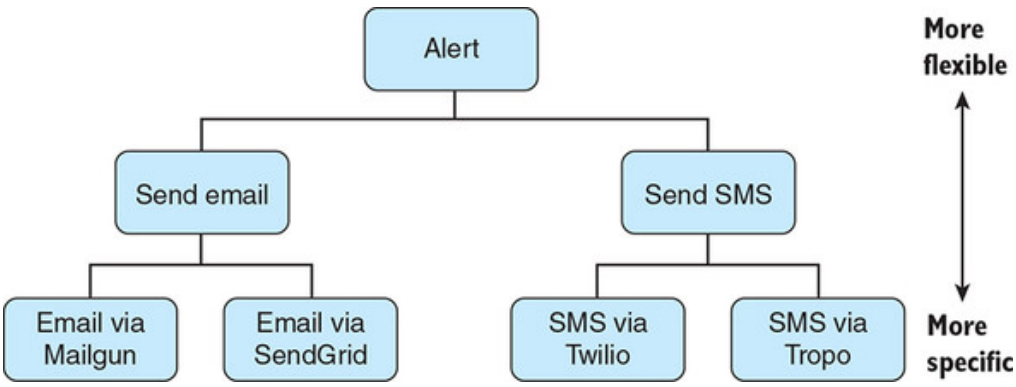
- Having a separate command executor for all high- or medium-priority command types, so that each command executor operates (and fails) independently of the others.
- Copying all commands into a separate publish-subscribe message queue, such as NSQ (see chapter 1) or SQS, with a scalable pool of workers to execute the commands. This queue would not have the Kafka or Kinesis notion of ordered processing, so there would less scope for functioning command types to be "blocked" by failing commands.

9.5.3. Command hierarchies

In this chapter, we have argued in favor of creating commands that are *shrink-wrapped* and *fully baked*. A command executor should have discretion as to how each command should be best executed, but the executor should not have to look up additional data or contain command-specific business logic. Following this design, we created an alert command for Plum, which was resolved by our executor into an email sent via Mailgun.

The alert command is a useful one, but under the hood our command executor was resolving it into a *send email* command, and then ultimately into a *send email via Mailgun* command. For your unified log, you may want to model these more granular commands as well as the high-level ones, so that a decision-making app can be as precise or flexible as it likes as to the commands it emits. Figure 9.11 illustrates this idea of a form of *command hierarchy*.

Figure 9.11. In a hierarchy of commands, each generation is more specific than the one above it. Under alert, we have a choice between alerting via email and alerting via SMS; within email and SMS, we have even more-specific commands to determine which provider is used for the sending.



Summary

- A command is an order or instruction for a specific action to be performed in the future. Each command, when executed, produces an event.
- Whereas most software uses implicit decision-making, apps in a unified log architecture can perform explicit decision-making that emits commands to one or more streams within the unified log.
- A stream of commands in a unified log can be executed using dedicated applications performing single-event processing.
- Commands should be carefully modeled to ensure that they are shrink-wrapped and fully baked. This ensures that decision-making and command execution can stay loosely coupled, with a clear separation of concerns.
- Commands can be modeled in Apache Avro or another schema tool (see chapter 6).
- Command executor apps can be implemented using simple Kafka consumer and producer patterns in Java.
- On successful execution of a command, the executor should emit an event recording the exact action performed.
- When scaling up an implementation of commands in your unified log, consider how many command streams to operate, how to handle execution failures, and whether to implement command hierarchies.